Lecture 5: Assembly and Boot

Professor G. Sandoval

05

NYU

Some slides adapted by G. Sandoval for CS3224, from slide by Brendan Dolan-Gavitt and Bryant and O'Hallaron, Computer Systems: A programmer's Perspective, Third Edition

Today’s route

- Recall the call-stack contract.
- Explain how BIOS starts xv6.
- Trace assembly setup and C kernel loading.

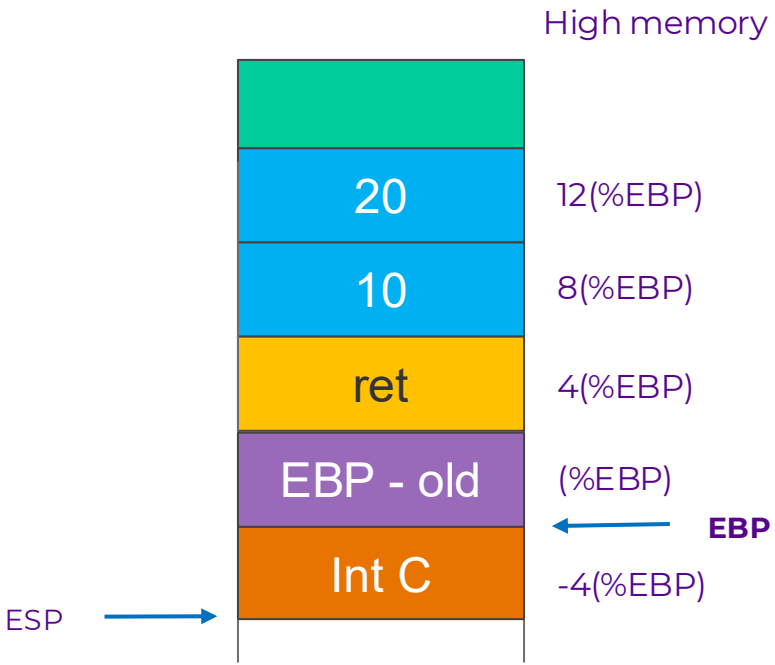
NYU

The call-stack contract

- A near call saves the next instruction’s address on the stack.
- A conventional prologue saves %ebp, then sets %ebp = %esp.
- Local allocation moves %esp. The frame pointer stays fixed.
- Our 32-bit cdecl caller removes stack arguments after ret.

NYU

A conventional stack frame



After this prologue, which operand reads the first argument

```
pushl %ebp
movl  %esp, %ebp
subl  $8, %esp
```

- 4(%ebp) 0%
- 8(%ebp) 0%
- (%esp) 0%
- 4(%esp) 0%
- None of the above 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Pair trace: caller cleanup

- Initially %esp = 0x8000. AddMe returns a + b in %eax.
- What are %eax and %esp just after the last instruction?

```
pushl $20
pushl $10
call  AddMe
addl  $8, %esp
```

Pairs: trace the stack addresses for 90 seconds.

NYU

The question for today

- At power-on, no operating system is running.
- How does the processor reach xv6 main()?
- We will follow each handoff and identify the job of the code currently running.

At every stage: who is running, where is the code, and what happens next?

NYU

Firmware, bootloader, kernel

- **Firmware** begins running when the machine starts. BIOS is the firmware in this example.
- The **bootloader** prepares the machine and places the kernel in memory.
- The **kernel** initializes the operating system and creates processes.
- Each stage prepares the conditions required by the next stage.

Three different programs, three different responsibilities.

A call needs a stack

- A 32-bit **call** subtracts four from %esp and stores the return address.
- The callee begins only after that write succeeds.
- **ret** reads the saved address and adds four to %esp.
- Calling C therefore requires a usable stack.

This familiar rule explains one job of the assembly bootstrap.

In 32-bit mode, %esp = 0x8000 just before call f. What happens before f executes its first instruction?

Return address stored at 0x7ffc; %esp = 0x7ffc

0%

Return address stored at 0x8000; %esp = 0x8004

0%

Argument stored at 0x7ffc; %esp = 0x7ffc

0%

Nothing touches the stack until f creates a frame

0%

None of the above

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

The complete boot story

1. **BIOS** loads the boot block at 0x7c00.
2. **bootasm.S** prepares the machine and calls **bootmain**.
3. **bootmain.c** copies the kernel from disk into memory and enters it.
4. **Kernel** entry prepares the kernel environment and calls main.
5. The **kernel** eventually starts the first user program, /init.

This sequence is the roadmap for the rest of the lecture.

Copying code and transferring control

- Copying places program bytes in memory. It does not start the program.
- Transferring control changes the processor's next instruction address.
- BIOS first copies the boot block, then starts it.
- bootmain first copies the kernel, then enters it.

Every handoff needs both a destination in memory and a control transfer.

Stage 1: BIOS

- Who runs? BIOS firmware.
- Where is the next code? BIOS copies the first disk sector to physical address 0x7c00.
- What happens next? BIOS transfers control to the boot block.
- The kernel is still on disk.

Handoff: BIOS to bootasm.S.

Prediction: just after BIOS

BIOS has transferred control to 0x7c00.

- Which code is executing now?
- Where is the kernel?
- Has any process been created?

Pairs: answer all three questions in 60 seconds.

NYU

Answer: the boot block runs first

- bootasm is executing from the boot block at 0x7c00.
- The kernel remains in its executable file on disk.
- No process exists because the kernel has not started.
- The next job is to create an environment where bootmain can run.

Current position: BIOS finished; the bootloader now runs.

Stage 2: bootasm

- Who runs? Assembly in the boot block.
- Job: create the environment required by 32-bit C.
- bootasm selects the processor mode and establishes a usable stack.
- Next: call bootmain.

Handoff: bootasm.S to bootmain.c.

NYU

bootasm's contract with C

- The processor can execute the 32-bit instructions produced by the compiler.
- Code and data addresses behave as bootmain expects.
- %esp points to writable memory before call bootmain.
- The exact chipset setup is an implementation detail, not today's learning goal.

bootasm creates bootmain's starting assumptions.

A stack before the first C call

- The loader sets %esp to 0x7c00 before call bootmain.
- The stack grows below the boot block.

```
64    # Set up the stack pointer and call into C.
65    movl    $start, %esp
66    call    bootmain
```

NYU

Pair trace: the first C call

- bootasm has set %esp = 0x7c00.
- It executes a 32-bit call to bootmain.
- At what address does call store the return address?
- What is %esp when bootmain begins?

Pairs: sketch the first stack slot. You have 60 seconds.

NYU

Stage 3: bootmain

- Who runs? bootmain, a small C function.
- Kernel location: an executable file on disk.
- Job: copy the kernel's code and data into memory.
- Next: enter the kernel at its starting address.

Handoff: bootmain to kernel entry.

File on disk and image in memory

- On disk, the kernel is stored as an executable file.
- The file contains program bytes plus information about where they belong.
- In memory, those bytes occupy the locations expected by the kernel.
- bootmain converts the stored file into the runnable memory image.

The file is the recipe; the memory image is the prepared program.

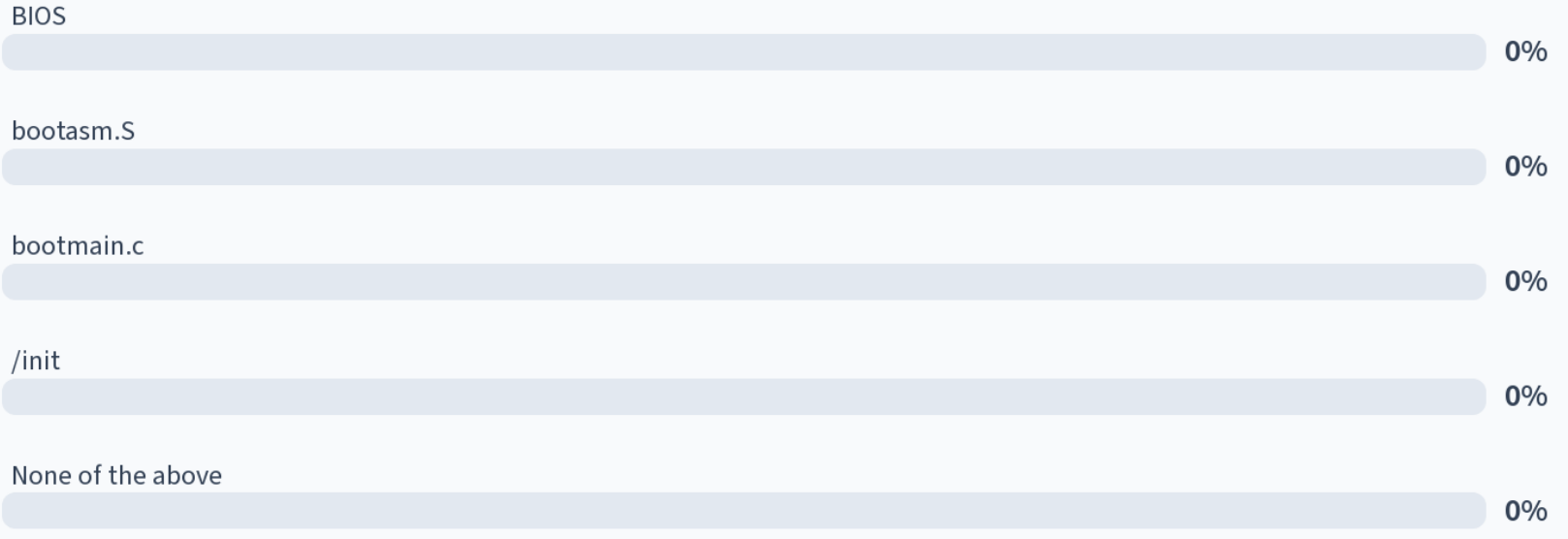
NYU

bootmain

```
read the executable description
for each kernel segment:
    copy its file bytes into memory
    zero any remaining memory
enter the kernel at its start address
```

Result: the kernel's memory image is ready to execute.

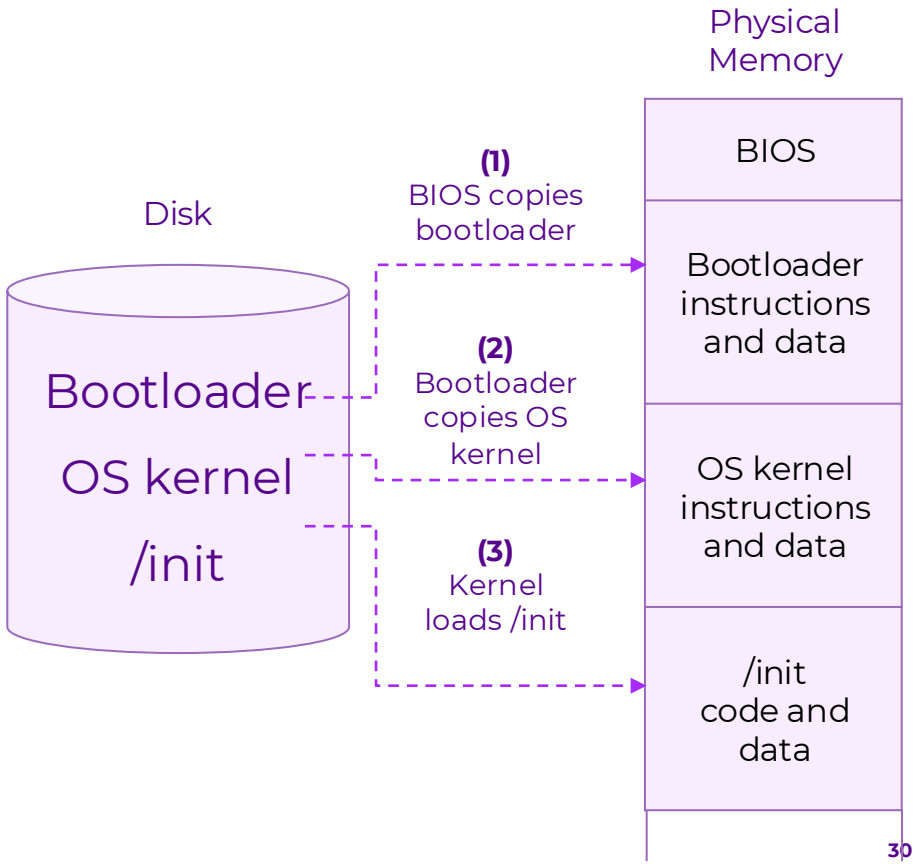
Which code copies the kernel's executable contents from disk into their required memory locations?



Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Answer: bootmain loads the kernel

- BIOS starts the bootloader.
- The bootloader loads and enters the kernel.
- The kernel later supports loading /init.



Stages 4 and 5

- Who runs? Kernel entry code, followed by kernel C.
- Kernel entry enables paging, selects a kernel stack, and reaches main.
- main initializes the operating system and creates the first process.
- The first user code asks the kernel to load /init.

Handoff: bootloader to kernel, then kernel to the first user program.

The first process comes later

- BIOS code runs without a process.
- The bootloader also runs before the kernel creates processes.
- Kernel entry and main run before the first user process.
- /init begins only after the kernel is running and can load a user program.

Running code does not always mean that a process exists.

Pair activity: arrange the handoffs

Place these events in order.

A. Kernel entry reaches main.
B. BIOS transfers control to 0x7c00.
C. bootmain enters the loaded kernel.
D. bootasm calls bootmain.
E. The first user code requests /init.

Pairs: order the letters and explain the middle handoff.

NYU

Failure reasoning

- Suppose bootmain begins, but the kernel entry code never runs.
- Which stages definitely completed?
- Which handoff failed?
- Could main or /init have run?

Pairs: reason from the sequence

Answer: the failure precedes the kernel

- BIOS completed its handoff because bootmain eventually began.
- bootasm completed enough setup to call C.
- The failure occurred while bootmain loaded the kernel or entered its start address.
- Kernel main and /init cannot have run.

The last observed stage narrows the failed handoff.

NYU

Exit Poll: Which sequence matches the xv6 boot path taught today?

BIOS, bootmain, bootasm, kernel entry, /init

0%

BIOS, bootasm, bootmain, kernel entry, /init

0%

bootasm, BIOS, kernel entry, bootmain, /init

0%

BIOS, kernel entry, bootasm, /init, bootmain

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**